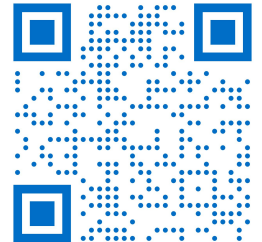


Андрей Державин,
Антон Шурыгин

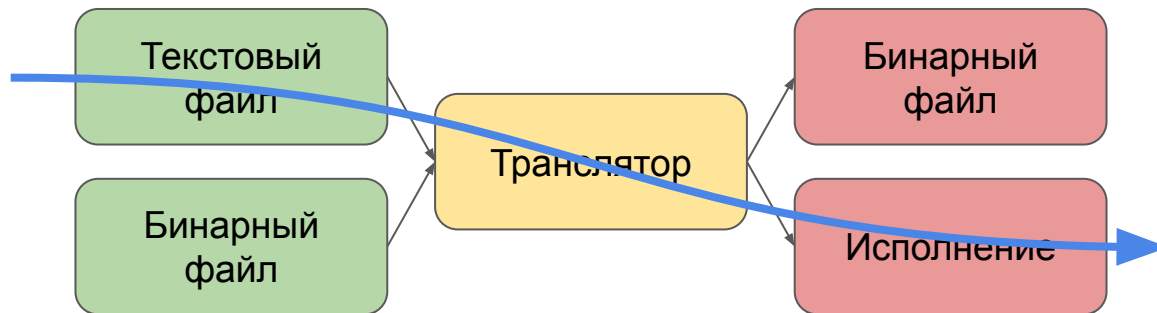
Симуляция ЦП и ОС

Лекция 2



Определение

- Что такое интерпретатор?

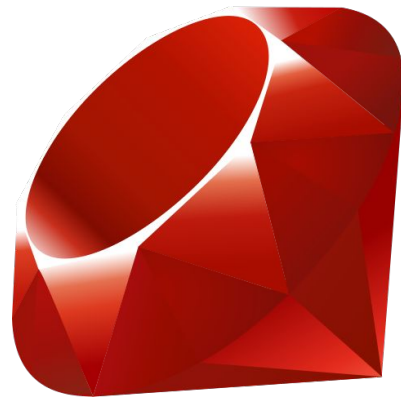
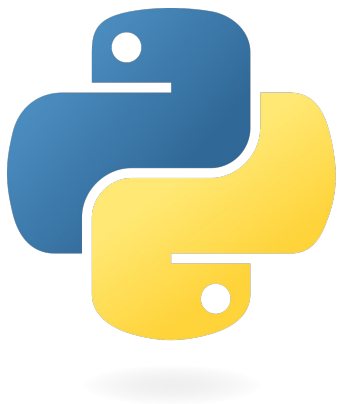


Определение

- Интерпретатор - один из видов трансляторов
- Какие существуют интерпретаторы?

Определение

- Интерпретатор - один из видов трансляторов
- Какие существуют интерпретаторы?



Интерпретатор

- Простейший подход симуляции
- Аналогия с интерпретаторами ЯП

```
a = 20  
b = 11  
b *= 2  
c = a + b
```

```
addi x1, x0, 20  
addi x2, x0, 11  
slli x2, x2, 1  
add x3, x1, x2
```

Интерпретатор

- Простейший подход симуляции
- Аналогия с интерпретаторами ЯП

```
a = 20
```

```
b = 11
```

```
b *= 2
```

```
c = a + b
```



```
addi x1, x0, 20
```

```
addi x2, x0, 11
```

```
slli x2, x2, 1
```

```
add x3, x1, x2
```

Интерпретатор

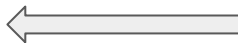
- Простейший подход симуляции
- Аналогия с интерпретаторами ЯП

```
a = 20
```

```
b = 11
```

```
b *= 2
```

```
c = a + b
```



```
addi x1, x0, 20
```

```
addi x2, x0, 11
```

```
slli x2, x2, 1
```

```
add x3, x1, x2
```

Интерпретатор

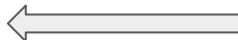
- Простейший подход симуляции
- Аналогия с интерпретаторами ЯП

```
a = 20
```

```
b = 11
```

```
b *= 2
```

```
c = a + b
```



```
addi x1, x0, 20
```

```
addi x2, x0, 11
```

```
slli x2, x2, 1
```

```
add x3, x1, x2
```

Интерпретатор

- Простейший подход симуляции
- Аналогия с интерпретаторами ЯП

```
a = 20
```

```
b = 11
```

```
b *= 2
```

```
c = a + b
```



```
addi x1, x0, 20
```

```
addi x2, x0, 11
```

```
slli x2, x2, 1
```

```
add x3, x1, x2
```

Интерпретатор

- Простейший подход симуляции
- Аналогия с интерпретаторами ЯП

```
a = 20  
b = 11  
b *= 2  
c = a + b
```



```
addi x1, x0, 20  
addi x2, x0, 11  
slli x2, x2, 1  
add x3, x1, x2
```

Интерпретатор

- Простейший подход симуляции
- Аналогия с интерпретаторами ЯП

```
a = 20  
b = 11  
b *= 2  
c = a + b
```



```
addi x1, x0, 20  
addi x2, x0, 11  
slli x2, x2, 1  
add x3, x1, x2
```

Интерпретатор

- Простейший подход симуляции
- Аналогия с интерпретаторами ЯП

```
a = 20  
b = 11  
b *= 2  
c = a + b
```



```
addi x1, x0, 20  
addi x2, x0, 11  
slli x2, x2, 1  
add x3, x1, x2
```

Интерпретатор

- Простейший подход симуляции
- Аналогия с интерпретаторами ЯП

```
a = 20  
b = 11  
b *= 2  
c = a + b
```



```
addi x1, x0, 20  
addi x2, x0, 11  
slli x2, x2, 1  
add x3, x1, x2
```

Интерпретатор

- Простейший подход симуляции
- Аналогия с интерпретаторами ЯП

```
a = 20  
b = 11  
b *= 2  
c = a + b
```

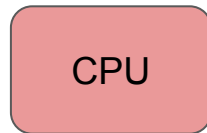
```
addi x1, x0, 20  
addi x2, x0, 11  
slli x2, x2, 1  
add x3, x1, x2
```

Программная модель

- Какой главный модуль в нашей модели?

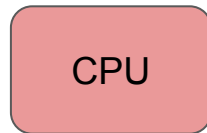
Программная модель

- Ядро исполняет инструкции



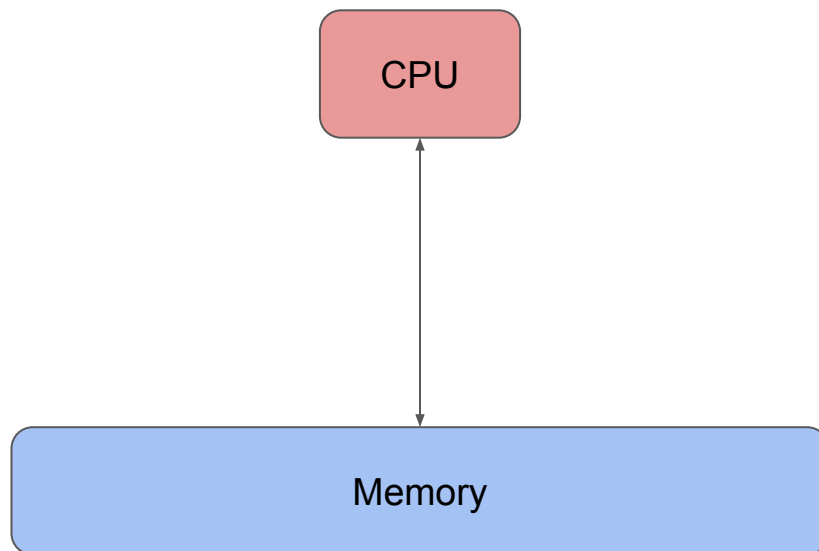
Программная модель

- Ядро исполняет инструкции
- Что ещё необходимо?



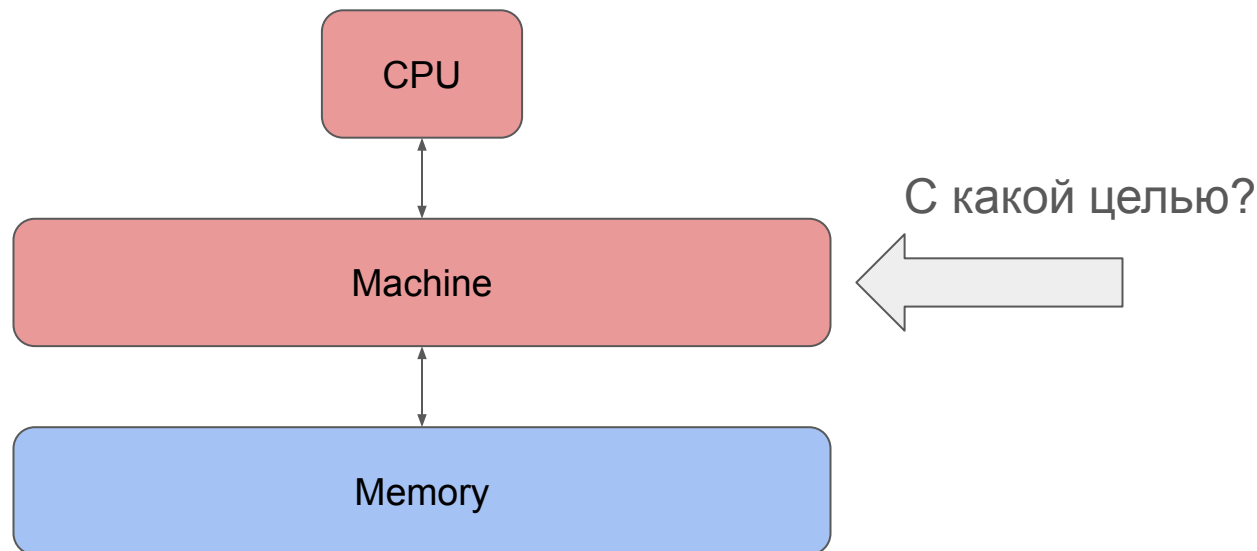
Программная модель

- Ядро исполняет инструкции
- Память - хранит код и данные



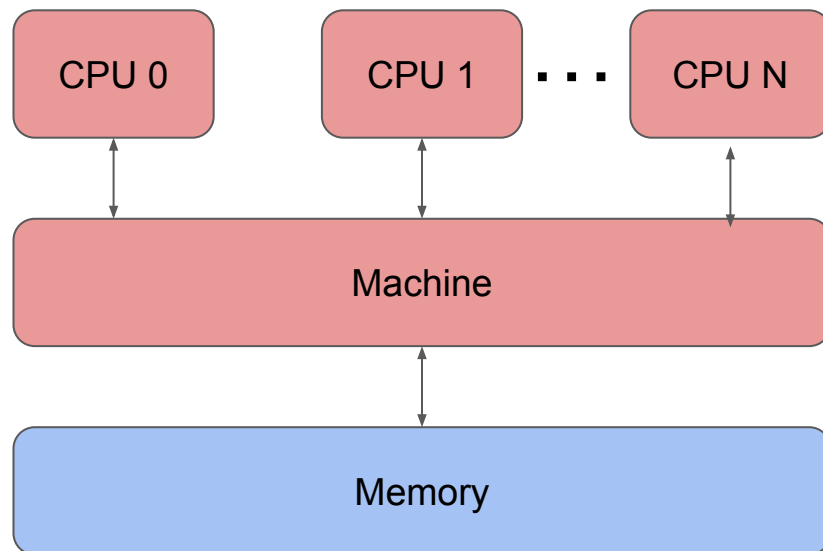
Программная модель

- Ядро исполняет инструкции
- Память - хранит код и данные



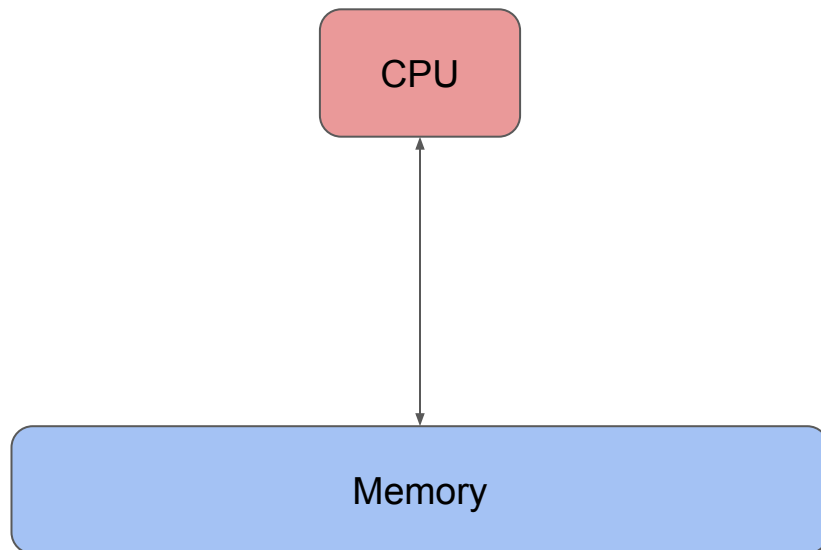
Программная модель

- Ядро исполняет инструкции
- Память - хранит код и данные



Программная модель

- Как бы вы реализовали простую модель?



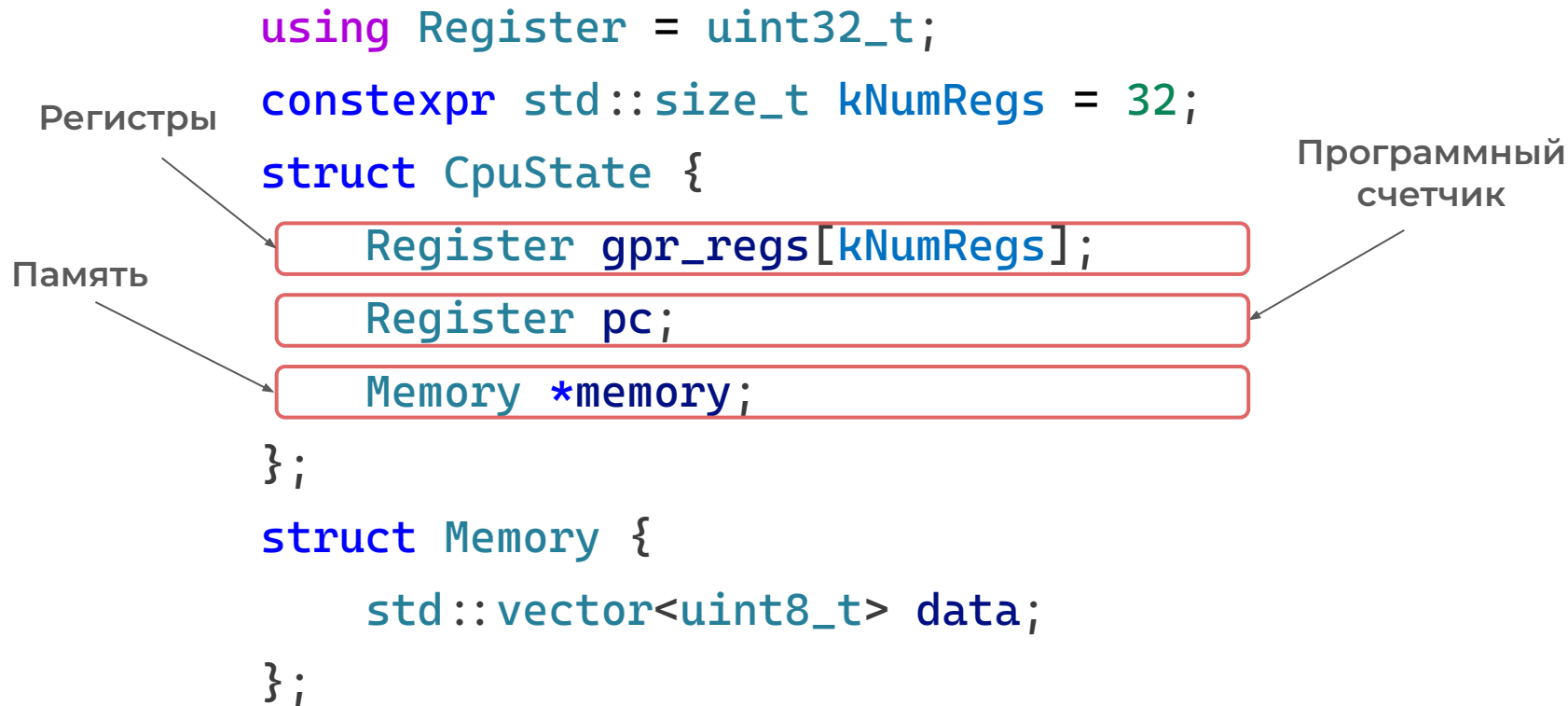
Простейшая модель на C++

```
using Register = uint32_t;
constexpr std::size_t kNumRegs = 32;
struct CpuState {
    Register gpr_regs[kNumRegs];
    Register pc;
    Memory *memory;
};
struct Memory {
    std::vector<uint8_t> data;
};
```

Регистры

Память

Программный счетчик



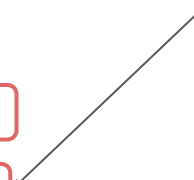
Простейшая модель на C++

```
using Word = std::uint32_t;
enum class Opcode : std::uint8_t {
    kUnknown = 0,
    kAdd,
    // ...
};
struct Instruction {
    Opcode opc{};
    Word src1{}, src2{}, dst{};
};
```

Код
операции

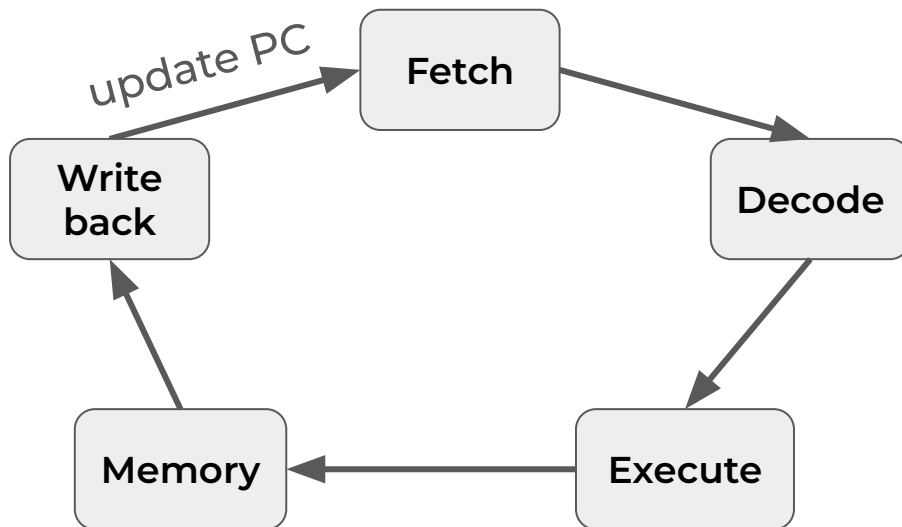


Операнды
инструкции



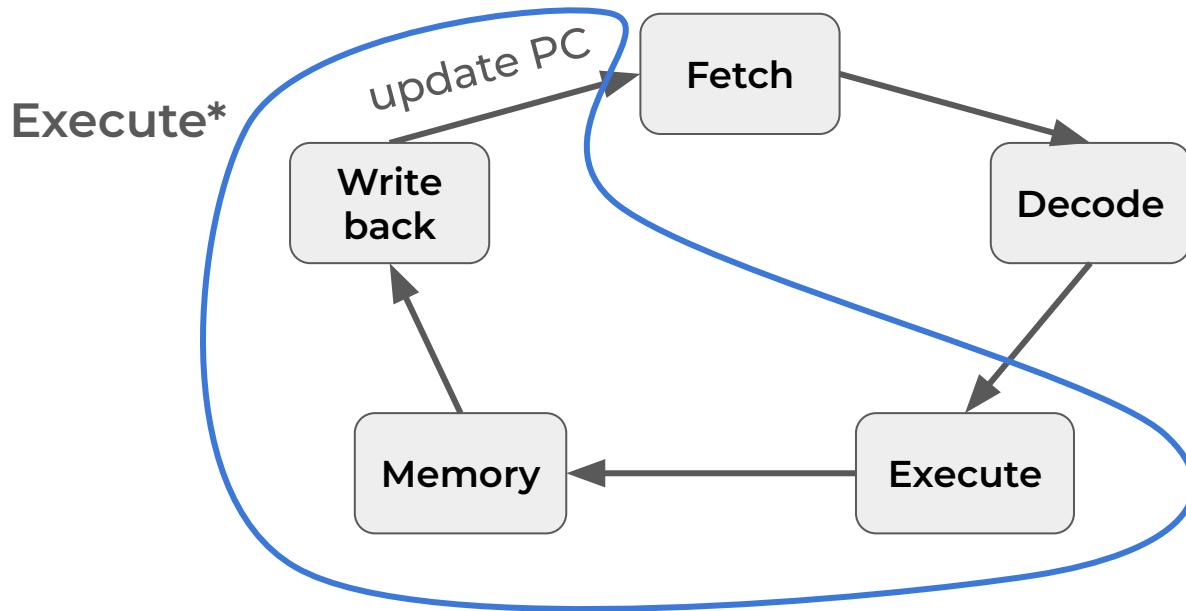
Интерпретатор

- CPU исполняет инструкции по стадиям
- Модели не нужно учитывать все детали



Интерпретатор

- CPU исполняет инструкции по стадиям
- Модели не нужно учитывать все детали



Простейшая модель на C++

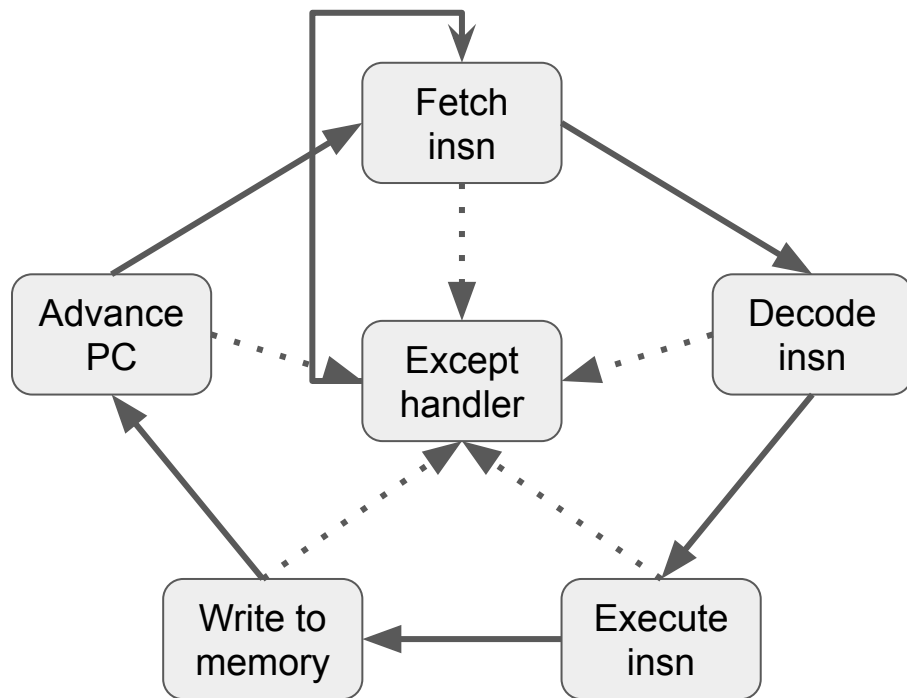
```
Instruction decode(Word encoding) {  
    Instruction insn{.opcode = get_opcode(encoding)};  
    switch (insn.opcode) {  
        case Opcode::kAdd:  
            insn.src1 = get_src1(encoding);  
            insn.src2 = get_src2(encoding);  
            insn.dst = get_dst(encoding);  
            break;  
    }  
    return insn;  
}
```

Простейшая модель на C++

```
void execute(CpuState *cpu, Instruction insn) {  
    switch (insn.opcode) {  
        case Opcode::kAdd:  
            auto res = cpu->getReg(insn.src1) +  
                        cpu->getReg(insn.src2);  
            cpu->setReg(insn.dst, res);  
            break;  
    }  
}
```

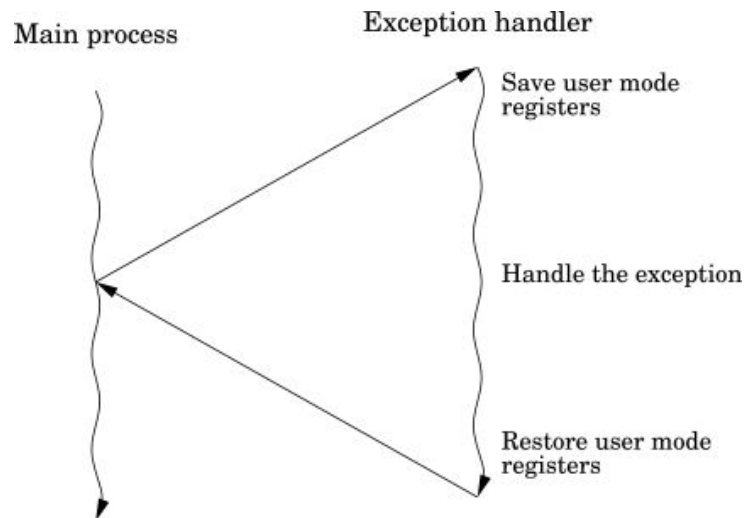
Исключения

- Исполнение может прерываться
 - Исключения случаются...
- Синхронные/асинхронные
- Могут требовать повторения инструкции
- Зависят от внешних событий



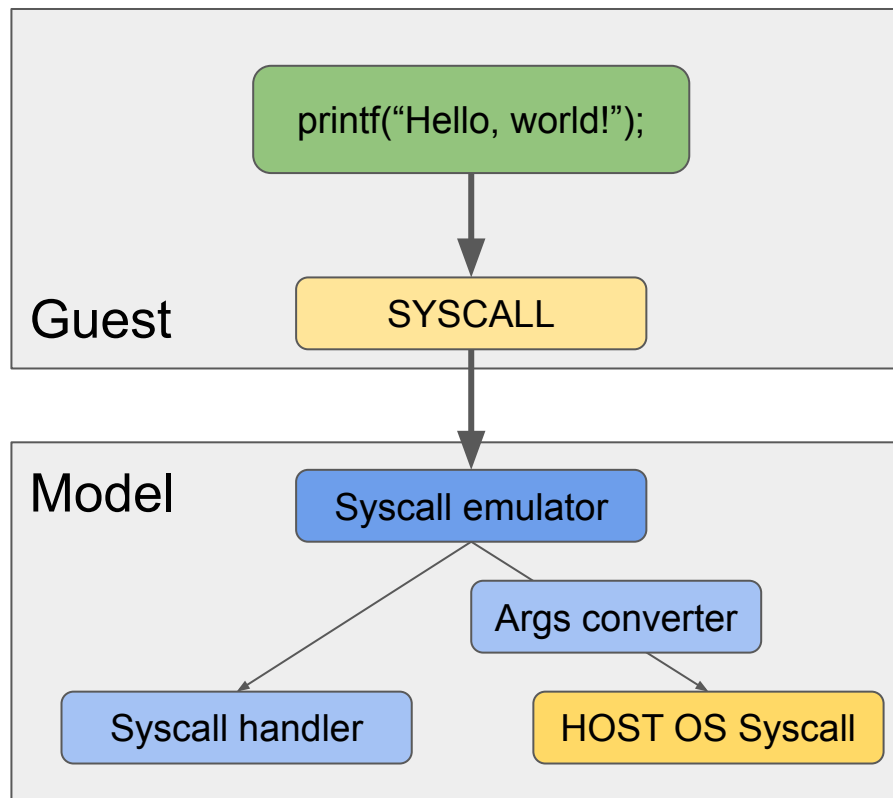
Исключения

- Как обрабатывать исключения?
 - Проверяем флаг исключения после каждой инструкции
- Как часто встречаются исключения?
 - Достаточно редко
 - Частая проверка, которая срабатывает редко
- Как обрабатывать исключения? x2
 - Требуется нелокальная передача управления
- `setjmp()`, `longjmp()` - good old C-style
- C++ style - exceptions?



Semihosting

- Почти все пользовательские приложения взаимодействуют с ОС
- Загрузка ОС занимает время
 - Требуется периферия
- Как избежать моделирования всей ОС?
- Воспользуемся принципом абстракции



Бенчмарки

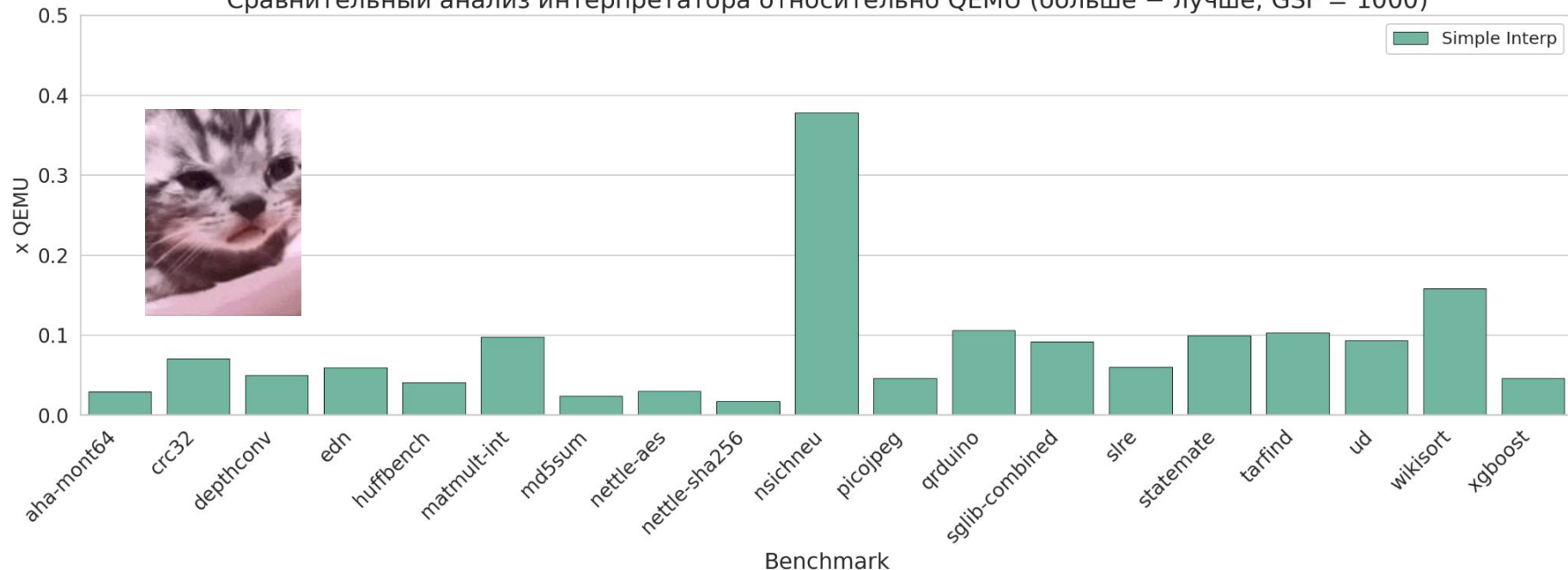


- embench-iot – набор тестов для встраиваемых систем на базе ARM Cortex-M

Тип вычислений	Тесты (Embench IOT 2.0)	Описание
Compute-bound	aha-mont64, crc32, nettle-sha256, md5sum, nettle-aes	Хеши, шифрование, CRC, целочисленное умножение
Memory-bound	matmult-int, statemate	Перемножение матриц, конечный автомат
Memory & Branching Intensive	sglib-combined, wikisort, slre, nsichneu	Структуры данных, сортировка, регулярные выражения, сетевой протокол
Balanced	edn, picojpeg, qrduino, ud, depthconv, xgboost	Свёртки, JPEG, QR, сжатие, поиск в архиве

Померимся?

Сравнительный анализ интерпретатора относительно QEMU (больше – лучше, GSF = 1000)



Ускорение интерпретатора

➤ Какие узкие места можно выделить в представленной модели?

```
Instruction decode(Word encoding) {
```

```
    Instruction insn{.opcode =
```

```
    get_opcode(encoding)};
```

```
    switch (insn.opcode) {
```

```
        case Opcode::kAdd:
```

```
            insn.src1 = get_src1(encoding);
```

```
            insn.src2 = get_src2(encoding);
```

```
            insn.dst = get_dst(encoding);
```

```
            break;
```

```
    }
```

```
    return insn;
```

```
}
```

```
void execute(CpuState *cpu, Instruction insn) {
```

```
    switch (insn.opcode) {
```

```
        case Opcode::kAdd:
```

```
            auto res = cpu->getReg(insn.src1) +
```

```
                        cpu->getReg(insn.src2);
```

```
            cpu->setReg(insn.dst, res);
```

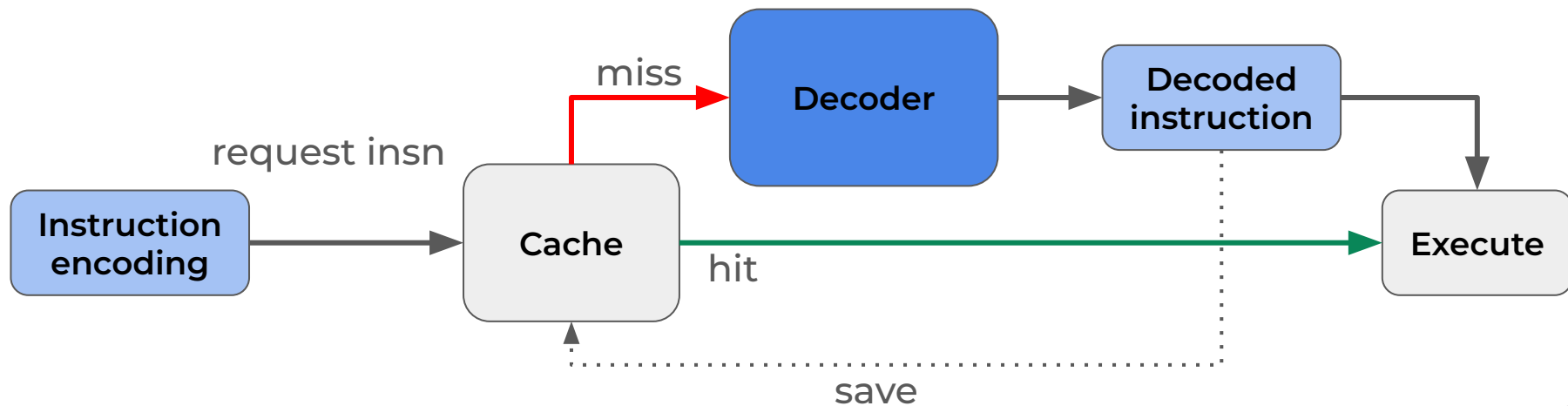
```
            break;
```

```
    }
```

```
}
```

Ускорение интерпретатора

- 80% времени работы занимают циклы
- Многие инструкции повторяются
- Повторно декодируем много инструкций
- Идея - кэшировать (запоминать) результаты декодирования



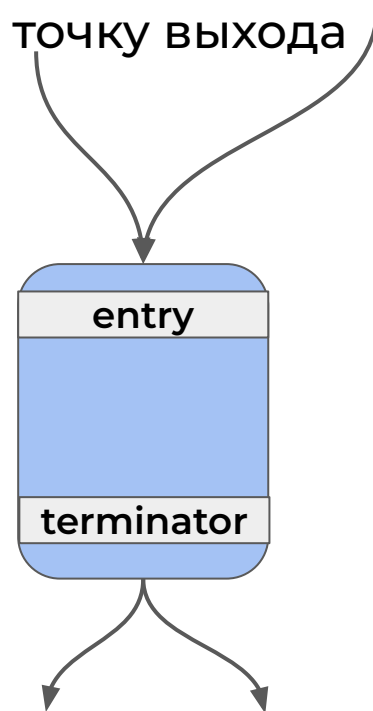
Ускорение интерпретатора

- Что происходит на каждой итерации цикла?
 - 3 поиска в кэше
 - 3 исполнения инструкции
- Что лишнее?

```
loop:
    add    x10, x10, x11
    addi   x11, x11, 1
    bge    x12, x11, loop
```

Ускорение интерпретатора

- *Линейный участок кода (Basic Block)* - последовательность инструкций, имеющая **одну** точку входа и **одну** точку выхода
 - Точка выхода - инструкция перехода
- Какое свойство следует?
 - ВВ выполняется только целиком
 - Нельзя выйти из середины
 - Нельзя зайти не в начале
 - ВВ - **неделимая единица исполнения**
- Можно кэшировать линейный участок целиком
 - Ключ - PC начала блока



Ускорение интерпретатора

- Найдите линейные участки в коде

```
loop:
```

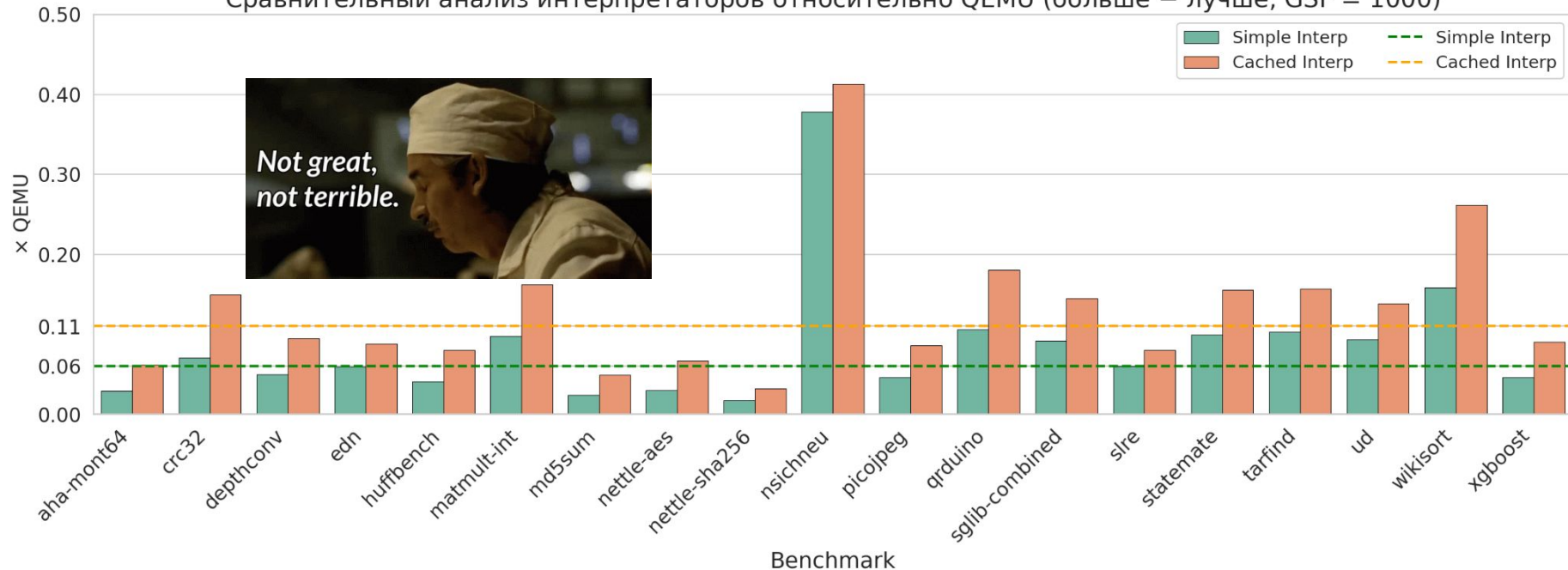
```
    add    x10, x10, x11  
    addi   x11, x11, 1  
    bge    x12, x11, loop
```



- Что происходит на каждой итерации цикла?
 - 1 поиск в кэше
 - 3 исполнения инструкции

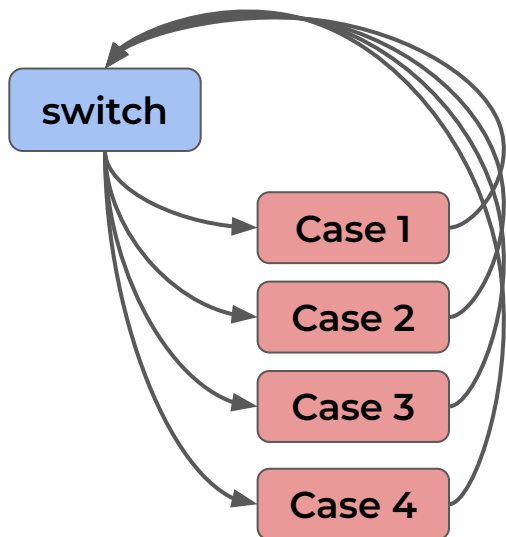
Померимся 2

Сравнительный анализ интерпретаторов относительно QEMU (больше – лучше, GSF = 1000)



Ускорение интерпретатора

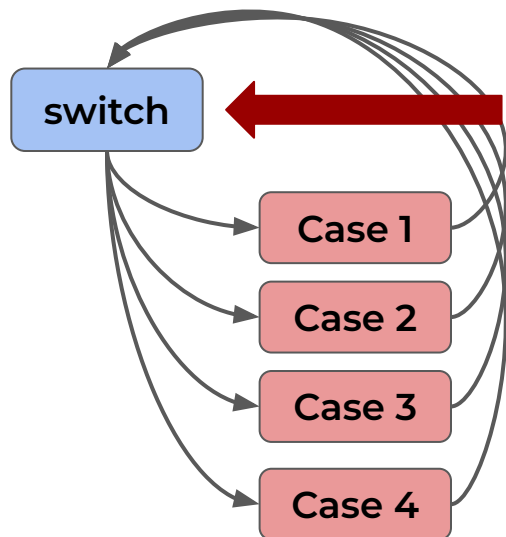
- Чем плох один большой switch?



```
void execute(...) {  
    switch ( ... ) {  
        case kAdd: doAdd(...); break;  
        case kSub: doSub(...); break;  
        case kMul: doMul(...); break;  
        case kDiv: doDiv(...); break;  
        ...  
    }  
}
```

Ускорение интерпретатора

- Чем плох один большой switch?



**Единая точка
диспетчеризации**

```
void execute(...) {  
    switch ( ... ) {  
        case kAdd: doAdd(...); break;  
        case kSub: doSub(...); break;  
        case kMul: doMul(...); break;  
        case kDiv: doDiv(...); break;  
        ...  
    }  
}
```

Ускорение интерпретатора

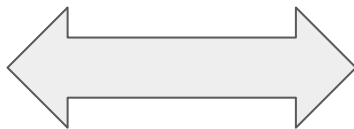
- Хозяйский CPU предсказывает адреса переходов
- Результаты хранятся в таблице
- Target предсказывается на основе истории исполнения

Branch PC	Prediction	Target
0x1010	taken	0x2000
0x1024	not taken	0x1028
0x4000	...	...

Ускорение интерпретатора

- Чем плох один большой switch?
- Один PC имеет большое число target'ов
 - Предсказания плохие
- Исполнение приостанавливается для коррекции

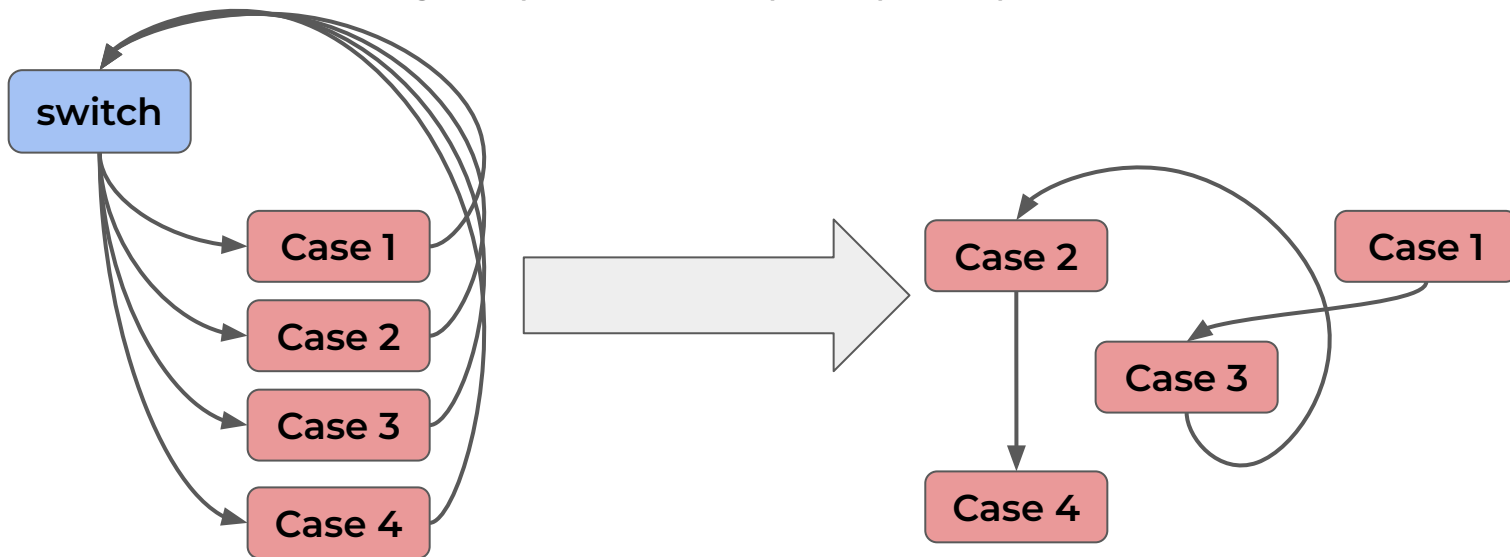
```
switch (x) {  
  case A: ...  
  case B: ...  
  case C: ..  
    ...  
}
```



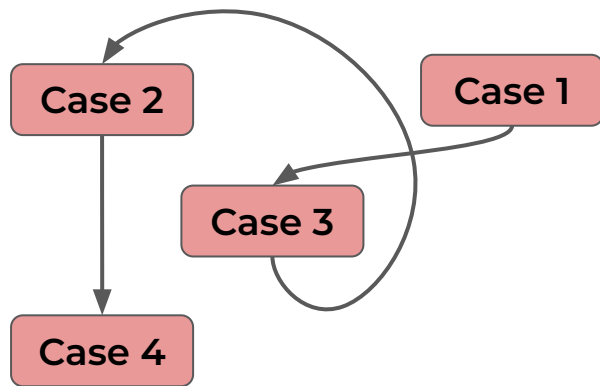
```
cmp    rdi, N  
ja     .L_default  
jmp    [.L_jumtable + rdi*8]
```

Ускорение интерпретатора

- Шитый код (threaded code)
- Введём несколько точек диспетчеризации
- Множество jump PC, история распределена



Ускорение интерпретатора



GNU Extension

```
DO_ADD:  
    doAdd(cpu, insn);  
    goto *label[++pc];
```

Указатели на функции

```
#define DISPATCH() \  
    [[clang::musttail]] \  
    return \  
    (handlers[memory[pc]]())()  
void doAdd() {  
    // ...  
    DISPATCH();  
}
```

Домашнее задание №1

- Безумный Архитектор
 - Реализовать симулятор ISA, которую спроектировал архитектор
 - В каждой СК набор из 14 различных инструкций
 - Кодировки между СК для одинаковых инструкций отличаются
 - В каждой СК есть несколько необычных RISC инструкций

Table of Contents

J	3
SYSCALL	4
STP	5
RORI	6
SLTI	7
ST	8
BDEP	9
CLS	10
ADD	11
BNE	12
BEQ	13
LD	14
AND	15
SSAT	16

Пример содержания спецификации

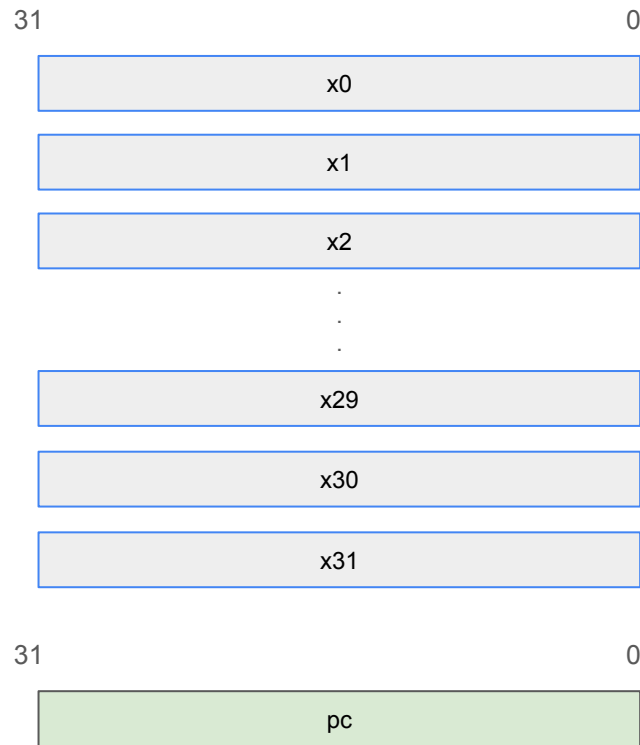
Домашнее задание №1



Домашнее задание №1

Возвращаясь к `struct CpuState`

- Регистровый файл
 - X – регистры общего назначения
 - Ширина регистра – 32 бита
 - Количество доступных регистров – 32
- PC – специальный 32-битный регистр, указывающий на исполняемую инструкцию



Регистровый файл

Литература

1. [Simulation Foundations Book, in Russian, Grigory Rechistov](#)
2. [The RISC-V Instruction Set, Manual Volume I, Unprivileged Architecture](#)
3. [НАШ Доклад на C++ Russia](#)